Pocket Plate

Phase-1.1

Team 6

Member Name	Member Roll #	Primary Responsibility
Manal Hameed	24L-0773	Identify Functional Clusters, Document Modules & Details,Research Available Technologies and update Jira
Izma Qamar	24L-0577	Assign Use Cases to Preliminary Modules & Compare & Evaluate Options for tech stack
Ayan Fatima	24I-0884	Refine Module Boundaries & check compatibility and assess feasibility tp finalise tech stack

Tech Stack:

The development of the PocketPlate system requires a modern and scalable technology stack capable of supporting dynamic user interfaces, real-time interactions, and efficient data management. The following technologies have been selected to ensure reliability, ease of development, and scalability.

4.1 Frontend Technologies

The frontend of the system will be developed using **React**, a popular JavaScript library used for building dynamic and interactive user interfaces. React enables the application to be structured using reusable components, which improves maintainability and scalability.

For styling and layout design, **Tailwind CSS** will be used. Tailwind CSS is a utility-first CSS framework that allows developers to rapidly build modern and responsive interfaces without writing large amounts of custom CSS.

Navigation between different pages of the application will be managed using **React Router**, which enables seamless client-side routing and improves user experience by preventing full page reloads.

Additionally, application state management will be handled using **Redux Toolkit**, which helps manage shared data such as the shopping cart, user session, and recommendations across different components of the application.

4.2 Backend Technologies

The backend of the system will be built using **Node.js**, a JavaScript runtime environment designed for building scalable server-side applications. Node.js is well suited for handling multiple requests efficiently and integrates well with modern frontend frameworks.

To simplify the development of server-side APIs, the **Express.js** framework will be used. Express provides a lightweight and flexible structure for creating RESTful APIs that manage authentication, order processing, voucher handling, and recommendation logic.

For implementing real-time features such as group ordering, **Socket.IO** will be used. Socket.IO allows real-time communication between the client and server, enabling users to see updates instantly when group members add or modify items in a shared order.

4.3 Database

The system will use **MongoDB** as its primary database. MongoDB is a NoSQL database that stores data in flexible document-based structures, making it suitable for applications with evolving data models such as food menus, user preferences, vouchers, and orders.

To facilitate interaction between the backend and the database, **Mongoose** will be used. Mongoose provides a structured way to define schemas, validate data, and perform database operations within the Node.js environment.

The database will store information such as:

- User accounts and authentication details
- Restaurant and menu data
- Orders and group orders
- Vouchers and promotional offers
- Reviews and ratings

4.4 Authentication and Security

User authentication will be implemented using **JSON Web Token**. JWT allows secure communication between the client and server by generating a token after successful login. This token is then used to verify user identity when accessing protected resources such as order placement or voucher redemption.

This approach ensures secure session management without requiring server-side session storage.

4.5 Deployment and Hosting

The application will be deployed using modern cloud hosting platforms to ensure accessibility and scalability.

The frontend application will be hosted on **Vercel**, a platform optimized for deploying frontend frameworks like React. Vercel provides fast global content delivery and automatic deployments from version control systems.

The backend server will be deployed using **Render or Railway**, both of which provide easy deployment for Node.js applications and support continuous integration workflows.

The database will be hosted using **MongoDB Atlas**, a fully managed cloud database service that provides high availability, security, and scalability.

4.6 Development Tools

To support development and collaboration, the following tools will be used:

- **Git** for version control
- **GitHub** for repository management and collaboration
- **Postman** for testing backend APIs
- **Visual Studio Code** as the primary code editor

These tools help maintain code quality, enable collaborative development, and streamline testing and deployment processes.

Technology Stack Overview

The **PocketPlate** system will be developed using the following technologies:

- **Frontend Framework:** React — used to build the user interface and create dynamic components for pages such as restaurant listings, cart, and recommendations.

- **Styling Framework:** Tailwind CSS — used to design responsive and modern user interfaces efficiently.

- **Routing:** React Router — used to manage navigation between different pages of the application.

- **State Management:** Redux Toolkit — used to manage shared application data such as user sessions, cart items, and order information.

- **Backend Runtime:** Node.js — used to run server-side logic and handle client requests.

- **Backend Framework:** Express.js — used to build RESTful APIs for features such as authentication, order processing, and recommendation logic.

- **Real-Time Communication:** Socket.IO — used to enable real-time updates for features like group ordering.

- **Database:** MongoDB — used to store application data including users, restaurants, menus, orders, and vouchers.

- **Database ODM:** Mongoose — used to interact with MongoDB and define structured data models.

- **Authentication:** JSON Web Token — used to securely authenticate users and manage sessions.

- **Frontend Deployment:** Vercel — used to host and deploy the frontend application.
- **Backend Deployment:** Render or Railway — used to host the backend server.
- **Database Hosting:** MongoDB Atlas — used for cloud-based database hosting.
- **Version Control:** Git and GitHub — used for source code management and collaboration.
- **API Testing:** Postman — used for testing backend APIs.
- **Development Environment:** Visual Studio Code — used as the primary code editor.

Modules:

Module 1:

Restaurant Browsing & Offers

Handles restaurant discovery and promotional information.

Use Cases:

- UC-10 – Browse Restaurants
- UC-11 – View Restaurant Detail
- UC-12 – View Newly Opened Restaurants
- UC-13 – View Daily Updates Page
- UC-14 – View Promotions
- UC-15 – View Daily Offers
- UC-16 – View Vouchers
- UC-17 – View Student-Exclusive Vouchers
- UC-18 – View Card Based Discounts

Module 2:

User Profile & Preferences

Handles user information, preferences, and personal settings.

Use Cases:

- UC-03 – Manage Profile
- UC-04 – Set Preferences
- UC-05 – View Order History
- UC-06 – Add Reviews
- UC-19 – Enable/Set Location

Module 3:

Budget & Recommendation System

Handles budgeting features and AI-based meal recommendations.

Use Cases:

- UC-07 – Enter Budget
- UC-08 – Apply Budget Filter
- UC-09 – Activate End-Of-Month Mode
- UC-20 – Get Optimised Meal Recommendation
- UC-22 – Get Group Recommendation & Bill Split
- UC-23 – Add Members
- UC-24 – Enter Group Budget
- UC-25 – Customise Individual Share

Module 4:

Ordering & Payment System

Handles the ordering workflow and checkout.

Use Cases:

- UC-21 – Build Cross-Restaurant Combo
- UC-26 – Select Payment Card
- UC-27 – Add Items to Cart
- UC-28 – Manage Multi-Restaurant Cart
- UC-29 – Place Order
- UC-30 – Make Payment
- UC-31 – Apply Vouchers

Module 5:

Authentication & Admin Management

Handles account access and system administration.

Use Cases:

Authentication

- UC-01 – Register Account
- UC-02 – Login

Admin Management

- UC-32 – Manage Users
- UC-33 – Manage Restaurants
- UC-34 – Manage Restaurant Integration
- UC-35 – Update Offers and Promotions Data
- UC-36 – Monitor System Data